Memorandum: Loan comparison, simple versus compound interest

Marker's copy | 25 marks | Model file: `loan_comparison.py` (run in Thonny on all three test sets)

Marks by outcome

Outcome	Parts	Marks
O1. Plan a solution using an IPO chart	Plan	4
O2. Write a Python program using input, conversion, arithmetic and formatted output	Input and conversion, Calculations, Output, Documentation	14
O3. Test a program, then find, explain and correct errors	Testing, Reasoning question	7
Total		**25**

Model solution (`loan_comparison.py`)

```
# Program: Loan comparison - simple interest versus compound interest
# Author: Your Name
# Date: 22 October 2026
# Purpose: Compare the amount paid back by a borrower charged simple interest
#          with a borrower charged compound interest (compounded annually)
#          on the same loan, rate and period.

# --- Input and conversion ---
simple_name = input("Borrower paying simple interest: ").strip().title()
compound_name = input("Borrower paying compound interest: ").strip().title()
loan_text = input("Loan amount in rand (e.g. R1 500,00): ")
rate_text = input("Annual interest rate in % (e.g. 6,5%): ")
months = int(input("Loan period in months (e.g. 36): "))

principal = float(loan_text.replace("R", "").replace(" ", "").replace(",", "."))
rate = float(rate_text.replace("%", "").replace(",", ".").strip()) / 100
years = months / 12

# --- Calculations ---
simple_amount = principal * (1 + rate * years)
compound_amount = principal * (1 + rate) ** years
difference = compound_amount - simple_amount
percent_difference = difference / simple_amount * 100

# --- Output ---
print()
print("LOAN COMPARISON")
print(f"{simple_name} (simple interest) pays back: R{simple_amount:.2f}")
print(f"{compound_name} (compound interest) pays back: R{compound_amount:.2f}")
print(f"Difference (compound - simple): R{difference:.2f}")
print(f"Percentage difference: {percent_difference:.2f}%")
```

Model IPO chart (Part A)

Inputs (with unit and type)	Processing (formulas)	Outputs (with unit)
Simple borrower's name (text)	principal = loan text cleaned, then converted to a decimal number	Both names (text)
Compound borrower's name (text)	rate = rate text cleaned, converted, then ÷ 100	Simple amount (R, 2 decimals)
Loan amount (text, rand, e.g. R1 500,00)	years = months ÷ 12	Compound amount (R, 2 decimals)
Annual interest rate (text, % per year, e.g. 6,5%)	simple amount = principal × (1 + rate × years)	Difference (R, 2 decimals)

Inputs (with unit and type)	Processing (formulas)	Outputs (with unit)
Loan period (whole number, months)	compound amount = principal × (1 + rate) ^ years	Percentage difference (%, 2 decimals)
	difference = compound amount − simple amount	
	percentage difference = difference ÷ simple amount × 100	

Expected outputs

Test 1 (`neo`, `lebo`, `R1 500`, `6,5%`, 36)

```
LOAN COMPARISON
Neo (simple interest) pays back: R1792.50
Lebo (compound interest) pays back: R1811.92
Difference (compound - simple): R19.42
Percentage difference: 1.08%
```

Test 2 (`ayanda`, `BONGANI`, `R10 000,00`, `12,5 %`, 60)

```
LOAN COMPARISON
Ayanda (simple interest) pays back: R16250.00
Bongani (compound interest) pays back: R18020.32
Difference (compound - simple): R1770.32
Percentage difference: 10.89%
```

Test 3 (`zanele`, `ITUMELENG`, `R5 000`, `10%`, 6)

```
LOAN COMPARISON
Zanele (simple interest) pays back: R5250.00
Itumeleng (compound interest) pays back: R5244.04
Difference (compound - simple): R-5.96
Percentage difference: -0.11%
```

Working for Test 1: simple = 1500 × (1 + 0,065 × 3) = R1792.50; compound = 1500 × $1{,}065^3$ = R1811.92.

Marking guide (25 marks)

Each mark is awarded once, for evidence the marker can see in the file, the chart or the output.

#	Part	Outcome	What earns the mark	Mark
1	Plan (IPO)	O1	All five inputs listed, each with its unit or example (rand, %, months)	1
2	Plan (IPO)	O1	Data type stated for each input (text, text, text, text, whole number)	1
3	Plan (IPO)	O1	Formulas written for simple and compound amount, plus the conversions (months to years, % to decimal)	1
4	Plan (IPO)	O1	Outputs listed with units and 2 decimal places	1
5	Input and conversion	O2	All five prompts tell the user the unit or format	1
6	Input and conversion	O2	Loan amount: R, spaces and comma removed or replaced, then converted to a decimal number	1
7	Input and conversion	O2	Rate: % and comma handled, converted to a decimal number and divided by 100	1
8	Input and conversion	O2	Months converted to a whole number and divided by 12 to give years	1
9	Input and conversion	O2	Names cleaned with `.strip()` and `.title()` **and** meaningful variable names used for all inputs (both needed)	1

#	Part	Outcome	What earns the mark	Mark
10	Calculations	O2	Simple amount: principal × (1 + rate × years), using variables	1
11	Calculations	O2	Compound amount: principal × (1 + rate) ** years, using variables	1
12	Calculations	O2	Difference = compound − simple (in this order)	1
13	Calculations	O2	Percentage difference = difference ÷ simple × 100	1
14	Output	O2	Layout matches: heading, then the four lines in the required order	1
15	Output	O2	Amounts and percentage printed to exactly 2 decimal places (e.g. `R5250.00`)	1
16	Output	O2	Units shown: R before each amount and % after the percentage	1
17	Output	O2	Fixed text matches the layout: heading in capitals, labels spelled exactly as given	1
18	Documentation	O2	Header comment with program name or purpose, author and date	1
19	Testing	O3	Actual output recorded for all three tests	1
20	Testing	O3	Actual output compared with expected for all three tests (match, or the difference described)	1
21	Reasoning (a)	O3	States that `input()` returns text (a string), which cannot be added to a decimal number	1
22	Reasoning (a)	O3	Fix: convert with `float(...)`, e.g. `principal = float(input(...))`	1
23	Reasoning (b)	O3	Explains that multiplication is done before addition, so only `principal * 1` is multiplied and `rate * years` is just added on	1
24	Reasoning (b)	O3	Line rewritten with brackets: `principal * (1 + rate * years)`, giving R1792.50	1
25	Reasoning (c)	O3	Zanele (simple interest) pays more in Test 3 **and** the reason refers to the period being shorter than one year	1
	Total			**25**

Rules for markers

- A wrong formula loses only its own mark (10 to 13). Later lines are marked on the learner's own variables, and output marks (14 to 17) follow through if the layout and format are right.
- Accept different but meaningful variable names, and accept `(1 + rate) ** (months / 12)` or the conversion done at input or in the calculation.
- Mark 15 needs two decimals showing. `round()` alone does not do this (`R5250.0` earns no mark).
- A negative difference prints as `R-5.96`. This is correct for this task.
- If the program does not run, still mark the plan, the header comment and each line of code that is correct. Marks 14 to 17 need output, so award them from the learner's pasted output, if there is any.

Model answers: reasoning question

(a) `input()` always gives text. `"1500" * 1` is still text, and Python cannot add text to a decimal number (`rate * years`), so it raises a `TypeError`. Fix: `principal = float(input("Loan amount in rand: "))`. **(2)**

(b) Multiplication is done before addition, so the line works out `(principal * 1) + (rate * years)`, which is 1500 + 0.195, not 1500 × 1.195. Correct line: `simple_amount = principal * (1 + rate * years)`, which gives R1792.50. **(2)**

(c) Zanele (simple interest) pays back more in Test 3. The loan is only 6 months, which is less than one year. Over a period shorter than one year, compound interest grows more slowly than simple interest. The two are equal at exactly 12 months, and compound interest is higher after that. **(1)**

Short reflection

- **Cognitive levels:** tracing and explaining code (reasoning (a) and (b): understand), writing from a scenario (Part B: apply), finding and explaining a fault, and comparing the test results (reasoning (b) and (c): analyse).
- **Why these test sets:** the period changes (36, 60 and 6 months) while the formulas stay the same. This lets the learner see that compound interest is not always the larger amount, which is the point of the task and the reason Test 3 is there.
- **Where the task is demanding:** cleaning text typed with commas and spaces, converting months to years, and the bracket in the simple-interest formula. These are covered by marks 6 to 9 and 23 to 24.
- **Verified:** the model solution was run on all three test sets and every output matches the hand calculation.
- **Choice you can change:** the formulas are given in the task so that marks test the programming, not recall. Remove them to make the task harder.
- **Neighbour test (10 minutes): to be completed by you.** Questions the neighbour had to ask: _____. Changes made to the task: _____. (If they had to ask a question, rewrite the task.)